



Trabajo Práctico: REST API y Express

Objetivo:

Desarrollar una API REST utilizando **Node.js**, **Express**, **Prisma ORM** y **PostgreSQL** para dar soporte a la aplicación web desarrollada previamente en React.

Este trabajo práctico continúa el desarrollo iniciado en el TP de React. La aplicación frontend ya no deberá guardar los datos en localStorage, sino que deberá consumir una API propia desarrollada por el grupo.

El objetivo es que cada grupo construya el backend de su aplicación, modele la entidad principal elegida, implemente operaciones CRUD completas, conecte la API con una base de datos PostgreSQL y actualice el frontend para que trabaje con datos reales persistidos en una base de datos.

Este TP se corresponde con la etapa de construcción de un servidor API REST para dar soporte a la aplicación desarrollada previamente, retomando la modalidad grupal con tablero, planificación, seguimiento docente y rotación de roles dentro del equipo.

Modalidad de trabajo grupal:

Además del desarrollo técnico de la aplicación, cada grupo deberá organizar su trabajo utilizando un tablero tipo **Kanban** (por ejemplo, Trello, GitHub Projects, Notion, Linear u otra herramienta similar), en el que se visualicen las tareas pendientes, en proceso y finalizadas. En este trabajo práctico, uno de los integrantes deberá asumir un rol de coordinación, similar al de **PM / Scrum Master**, con responsabilidades vinculadas a la organización del tablero, el desglose del proyecto en tareas más pequeñas, la asignación y seguimiento del trabajo y la planificación general del desarrollo. Ese integrante podrá tener una carga relativamente menor de programación que sus compañeros, ya que también deberá ocuparse de la coordinación del equipo. El PM no puede ser el mismo integrante que cumplió el rol de PM en el TP 1 y en el TP 2.

Invitar a los profesores como miembros del equipo en el tablero.

Agustin: guillermo.chiarotto@est.fi.uncoma.edu.ar

Agustin GitHub: agustin.chiarotto

Lucas: lucas.margni@est.fi.uncoma.edu.ar

Lucas GitHub: lucasmargni-facu



Requerimientos

1) Crear un nuevo repositorio para el backend

Crear un repositorio nuevo en GitHub para el backend de la aplicación.

El repositorio debe contener únicamente el proyecto backend. El frontend continuará en su repositorio original.

En el archivo README.md del backend se deberá incluir obligatoriamente:

- Nombre del proyecto.
- Integrantes del grupo.
- Link al repositorio del frontend.
- Link al tablero Kanban.
- Descripción breve de la aplicación.
- Descripción de la entidad principal elegida.
- Instrucciones para instalar y ejecutar el backend localmente.
- Instrucciones para configurar las variables de entorno.
- Instrucciones para ejecutar migraciones.
- Instrucciones para ejecutar el seed.
- Link al deploy del backend en Vercel.
- Link al deploy del frontend actualizado, si corresponde.
- Capturas de pantalla o ejemplos de uso de la API.
- Cualquier otra información que consideren relevante.

2) Configurar un proyecto Node.js con Express

Crear una API utilizando Node.js y Express.

El proyecto deberá poder ejecutarse localmente con:

```
npm install  
npm run dev
```

También deberá contar con un script para producción:



```
npm start
```

La API deberá responder al menos una ruta de prueba o health check:

```
GET /api/health
```

Respuesta esperada:

```
{
  "status": "ok",
  "message": "API funcionando correctamente"
}
```

3) Configurar Prisma con PostgreSQL

El proyecto deberá utilizar Prisma ORM para conectarse a una base de datos PostgreSQL.

Deberán crear el archivo schema.prisma y definir un modelo correspondiente a la entidad principal de la aplicación.

La entidad debe ser la misma que eligieron para el TP de React.

El modelo deberá tener como mínimo:

- id
- campos propios de la entidad
- createdAt
- updatedAt

Ejemplo orientativo:

```
model Movie {
  id          Int      @id @default(autoincrement())
  title       String
  director    String
  year        Int
  genre       String
  rating      Int
  type        String
  watched     Boolean @default(false)
```



```
createdAt DateTime @default(now())  
updatedAt DateTime @updatedAt  
}
```

Cada grupo deberá adaptar el modelo a su propia entidad.

4) Crear y ejecutar migraciones

El proyecto deberá incluir migraciones de Prisma.

Deberán poder ejecutar:

```
npx prisma migrate dev
```

También deberá poder generarse el cliente de Prisma con:

```
npx prisma generate
```

El repositorio deberá incluir la carpeta de migraciones generada por Prisma.

No se aceptará un proyecto que funcione únicamente con cambios manuales en la base de datos.

5) Crear seed obligatorio

El proyecto deberá incluir un seed obligatorio para cargar datos iniciales en la base de datos.

El seed deberá crear varios registros de prueba de la entidad principal.

Se recomienda cargar al menos 20-30 registros iniciales como para que el scroll infinito y paginado se pueda apreciar.

El seed deberá poder ejecutarse con un comando documentado en el README, por ejemplo:

```
npx prisma db seed
```



Los datos cargados por el seed deben ser coherentes con la temática de la aplicación.

No se aceptará una API cuya base de datos quede vacía al iniciar el proyecto.

6) Implementar CRUD completo

La API deberá implementar CRUD completo sobre la entidad principal.

Deberán existir endpoints para:

```
GET /api/entidad
GET /api/entidad/:id
POST /api/entidad
PUT /api/entidad/:id
DELETE /api/entidad/:id
```

Reemplazar entidad por el nombre correspondiente.

Por ejemplo:

```
GET /api/movies
GET /api/movies/:id
POST /api/movies
PUT /api/movies/:id
DELETE /api/movies/:id
```

Cada endpoint deberá conectarse a la base de datos usando Prisma.

No se permite guardar datos en arrays en memoria.

No se permite simular la persistencia.

No se permite usar archivos JSON como base de datos.

7) Responder con códigos HTTP correctos

La API deberá responder con códigos HTTP adecuados según cada caso.

Casos esperados:

```
GET exitoso → 200
POST exitoso → 201
PUT exitoso → 200
DELETE exitoso → 200 o 204
Body inválido → 400
Recurso no encontrado → 404
Error inesperado del servidor → 500
```

Ejemplo de respuesta para un recurso no encontrado:

```
{
  "error": "Recurso no encontrado"
}
```

Ejemplo de respuesta para un body inválido:

```
{
  "error": "Datos inválidos",
  "details": [
    {
      "field": "title",
      "message": "El título es obligatorio"
    },
    {
      "field": "year",
      "message": "El año debe ser un número válido"
    }
  ]
}
```

8) Validar manualmente el body en POST y PUT

Los endpoints POST y PUT deberán validar obligatoriamente el body recibido.

La validación debe ser manual, es decir, realizada mediante funciones JavaScript.

No se deben utilizar librerías de validación como Zod, Joi, Yup, express-validator u otras similares.

Se debe validar, como mínimo:

- Que los campos obligatorios existan.
- Que los strings obligatorios no estén vacíos.
- Que los campos numéricos sean números válidos.
- Que los valores tengan sentido para la entidad.
- Que los campos con opciones limitadas respeten esas opciones.
- Que no se acepten objetos vacíos.
- Que se devuelva un mensaje claro cuando los datos no sean válidos.

Ejemplo:

```
const validateMovie = (body) => {  
  const errors = [];  
  
  if (!body.title || body.title.trim() === "") {  
    errors.push({  
      field: "title",  
      message: "El título es obligatorio",  
    });  
  }  
  
  if (!body.year || Number.isNaN(Number(body.year))) {  
    errors.push({  
      field: "year",  
      message: "El año debe ser un número válido",  
    });  
  }  
  
  return errors;  
}
```

Si el body no es válido, la API deberá responder con HTTP 400.

9) Separar responsabilidades en el código

El proyecto deberá estar organizado de manera clara.

Se recomienda una estructura similar a:

```
src
```

```
|— index.js
|— app.js
|— routes
|   |— entity.routes.js
|— controllers
|   |— entity.controller.js
|— services
|   |— entity.service.js
|— validations
|   |— entity.validation.js
|— prisma
|   |— prismaClient.js
|— middlewares
|   |— errorHandler.js

prisma
|— schema.prisma
|— seed.js
|— migrations
```

No es obligatorio que la estructura sea exactamente esta, pero sí se evaluará que el código esté ordenado y que no esté toda la lógica acumulada en un único archivo.

10) Conectar el frontend React con la API

El frontend desarrollado en el TP de React deberá ser actualizado para consumir la API creada en este trabajo práctico.

La aplicación React ya no deberá usar localStorage como mecanismo de persistencia.

El frontend deberá consumir la API para:

- Obtener el listado de elementos.
- Obtener el detalle de un elemento.
- Hacer búsquedas y filtrados.

Dependiendo el dominio y el proyecto, utilizar la aplicación React para o Postman/Swagger para:

- Crear un nuevo elemento.
- Editar un elemento existente.



- Eliminar un elemento.

Se puede seguir usando estado local de React para manejar formularios, filtros, carga, errores o estados temporales de la interfaz, pero los datos persistidos deben provenir de la API.

11) Manejar estados de carga y error en el frontend

El frontend deberá contemplar, al menos:

- Estado de carga mientras se obtienen datos de la API con algun loading spinner o texto.
 - Mensaje de error si la API falla.
 - Mensaje cuando no hay elementos cargados.
 - Mensaje cuando una operación de creación, edición o eliminación falla.
 - Actualización de la UI luego de una operación exitosa.
-

12) Configurar CORS

La API deberá permitir solicitudes desde el frontend.

Deberán configurar CORS correctamente para que el frontend pueda consumir el backend tanto en desarrollo como en producción.

Se recomienda utilizar variables de entorno para definir el origen permitido.

Ejemplo:

```
FRONTEND_URL=http://localhost:5173
```

En producción, deberán configurar la URL del frontend desplegado.



13) Variables de entorno

El proyecto deberá utilizar variables de entorno.

Deberán incluir un archivo `.env.example` con las variables necesarias, sin valores reales sensibles.

Ejemplo:

```
DATABASE_URL=  
PORT=3000  
FRONTEND_URL=
```

No se debe subir el archivo `.env` real al repositorio.

Por seguridad no se deben subir credenciales de Neon, Vercel ni ninguna otra plataforma a ningún repositorio.

Los `.env` se comparten por mensajería privada.

14) Deploy

El backend deberá estar desplegado en Vercel o en algún otro sistema de hosting.

La base de datos deberá estar creada en Neon usando PostgreSQL o algún otro sistema de hosting.

El deploy debe estar funcionando al momento de la corrección.

La API desplegada deberá conectarse correctamente a la base de datos de Neon.

El frontend deberá estar configurado para consumir la API desplegada.



Bonus opcional: Documentación con Swagger

Quien tenga ganas de profundizar y seguir avanzando con su proyecto backend, podrá documentar la API utilizando Swagger / OpenAPI.

La documentación deberá permitir visualizar los endpoints disponibles, los parámetros esperados, los bodies de ejemplo y las posibles respuestas.

Criterios de evaluación

Se tendrá en cuenta:

- Funcionamiento completo del CRUD.
 - Uso correcto de Node.js y Express.
 - Uso correcto de Prisma.
 - Conexión real con PostgreSQL.
 - Correcta definición del modelo de datos.
 - Existencia y funcionamiento de migraciones.
 - Existencia y funcionamiento del seed.
 - Validación manual del body en POST y PUT.
 - Uso correcto de códigos HTTP.
 - Manejo adecuado de errores.
 - Organización del proyecto.
 - Claridad del código.
 - Integración completa con el frontend React.
 - Eliminación del uso de localStorage como persistencia.
 - Deploy funcional del backend.
 - Base de datos funcional desplegada.
 - README claro y completo.
 - Trabajo grupal organizado mediante tablero Kanban.
 - Uso de Git y Pull Requests.
 - Participación equilibrada del equipo.
-



Modalidad de entrega

La entrega deberá incluir:

- Link al repositorio del backend.
- Link al repositorio del frontend.
- Link al tablero Kanban.
- Link al deploy del backend.
- Link al deploy del frontend actualizado, si corresponde.
- Base de datos de Neon configurada.
- Seed ejecutado.
- API funcionando en producción.
- Frontend consumiendo la API.

Buenas prácticas esperadas

El código debe:

- Estar separado en funciones cuando sea necesario.
- Evitar duplicación innecesaria.
- Usar nombres claros para variables y funciones.
- Responder con códigos HTTP adecuados.
- Mantener los datos en memoria durante la ejecución del servidor.
- Ser legible para otro desarrollador.